

CAMPUSPULSE – PROJECT SYNOPSIS

1. Introduction / Project Overview

Campus placement drives are the single most critical event in the professional academic calendar, yet they remain notoriously fragmented and chaotic. Placement officers, company recruiters, and final-year students currently navigate a broken ecosystem of manual spreadsheets, endless email threads, and unreliable verbal confirmations. This dysfunction manifests in several chronic failures:

- **Double-Booking Chaos:** Students are frequently scheduled for overlapping interviews with competing companies due to the absence of a centralized, real-time constraint-checking system.
- **Eligibility Ambiguity & Information Asymmetry:** Students waste valuable time applying for roles they are fundamentally unqualified for, while recruiters drown in hundreds of irrelevant applications—creating an immense cognitive load.
- **Poor Transparency & Communication:** Students miss critical interviews due to inadequate reminders, and Training & Placement Officers (TPOs) lack real-time data to track placement velocity.
- **The "Post-Interview Black Hole":** Most systems end at scheduling. Students are left in permanent limbo, never knowing if they were selected or rejected, and TPOs cannot accurately calculate final placement statistics because the recruitment loop is never formally closed.

To solve these challenges, this project presents **CampusPulse**—a full-stack, AI-enhanced placement automation system for Semester 1. It introduces a **Semantic Skill-Intelligence Engine** using Sentence-BERT embeddings to understand the *meaning* of a student's experience, an **Explainable AI (XAI) feedback module** to ensure transparent shortlisting, a **Multi-Stage Recruitment Pipeline** supporting Aptitude Tests, Group Discussions, and Interviews, and a **Complete Offer-to-Acceptance Lifecycle** that formally closes the recruitment loop with result declaration, offer letter uploads, and student acceptance/decline tracking. The system is built as a monolithic Node.js application with a Python microservice for the AI engine, deployed locally on a single server for a single institution.

2. Users of the System

The system serves three distinct user roles, each provisioned with a specialized dashboard and granular permissions:

User Role	Who They Are	Key Responsibilities
Student	Final-year undergraduate/postgraduate students actively seeking employment.	View semantically matched job drives, apply with one click, track real-time application status across the multi-stage pipeline, download .ics calendar invites, receive automated 24-hour reminders, view AI-generated feedback on their resume strengths/weaknesses, and formally accept or decline offer letters upon selection.
Recruiter	HR representatives and hiring managers from prospective companies.	Create detailed job postings with dynamic eligibility criteria, configure multi-stage pipelines (Aptitude/GD), view semantically shortlisted candidates with explainable AI justifications, schedule conflict-free tests/GDs/interviews, declare results at each stage, upload official offer letters (PDF), and view offer acceptance rates.
TPO (Training & Placement Officer)	College administrators responsible for managing the entire placement lifecycle.	Approve/reject recruiter registrations using the automated Trust Score, validate company credentials, bulk-import student data via CSV, monitor campus placement analytics including stage-wise funnel visualization and offer-to-acceptance ratios, and oversee all active drives.

3. Modules of the Project (Semester 1 – Mini Project)

Goal: Build a fully functional, interactive, and intelligent placement automation system with a multi-stage recruitment pipeline (Aptitude → GD → Interview), semantic AI shortlisting, conflict-free scheduling, and a complete offer-to-acceptance lifecycle—all running seamlessly on localhost.

SL.NO	Module Name	Description	Core Technology
1	Authentication & Authorization	Secure JWT-based login with strict role-based access control (Student/Recruiter/TPO). Accounts are pre-created by TPO via CSV; students activate via email link.	bcrypt, JWT, Express.js
2	Profile Management	Students upload structured resumes (PDF); Recruiters submit detailed company profiles; TPO manages institutional approvals.	Multer, MongoDB GridFS
3	Job Drive Management	Recruiters create job postings featuring CTC, application deadlines, and strict eligibility filters (CGPA, Backlogs, Branches). Recruiters can toggle Aptitude Test and/or Group Discussion stages.	Express.js, MongoDB
4	Verification Pipeline	A deterministic rules-engine that proactively blocks ineligible applicants <i>before</i> they can apply based on CGPA, branch, and backlogs—saving recruiter bandwidth.	Node.js logic layer
5	Application Engine with Multi-Stage Recruitment Pipeline	Students apply to eligible drives. The system tracks the entire comprehensive lifecycle: Applied → Under Review → Aptitude	MongoDB, Express.js

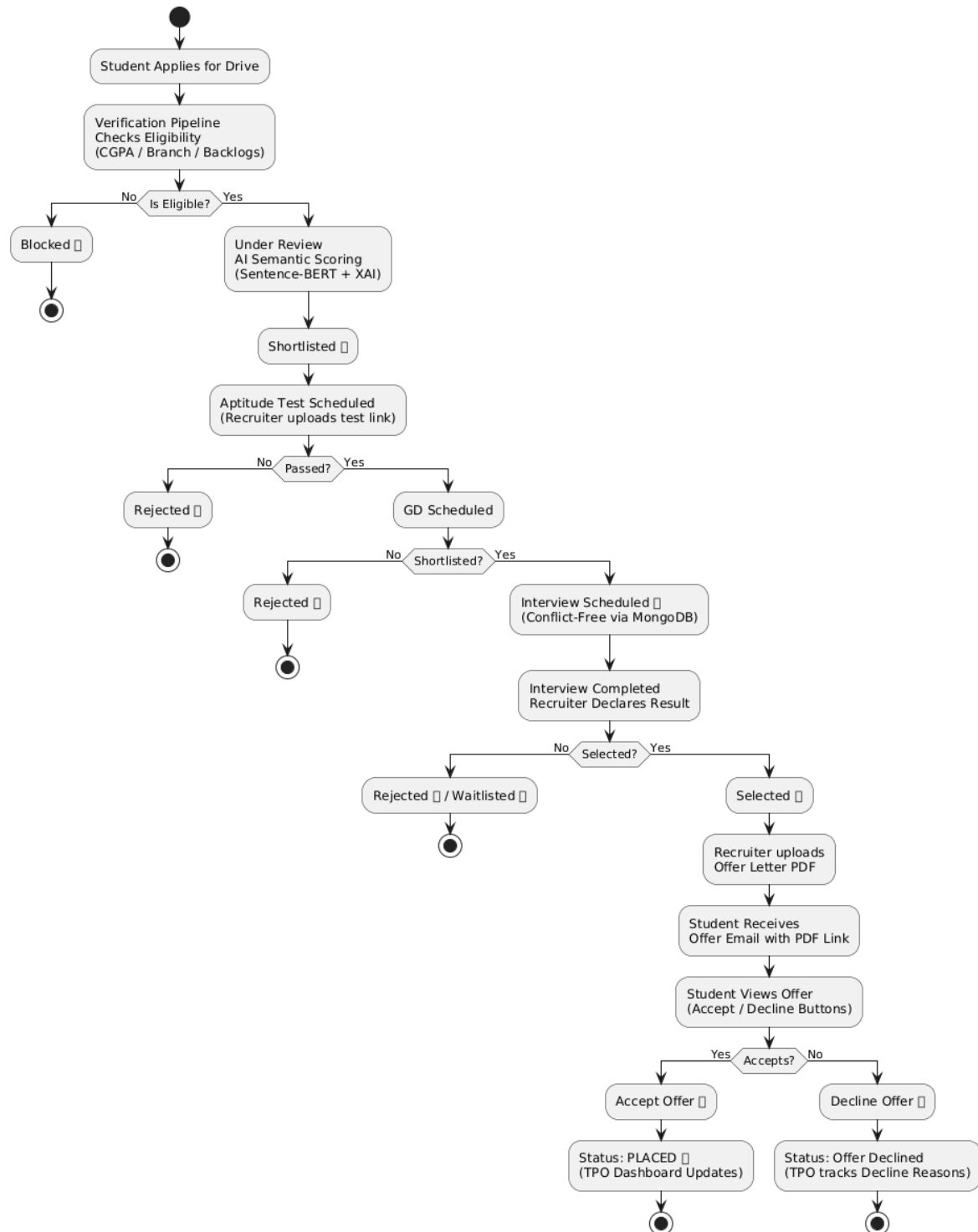
		Scheduled → Aptitude Completed → GD Scheduled → GD Completed → Interview Scheduled → Interview Completed → Selected/Rejected/Waitlisted → Offer Accepted/Declined → Placed. Stages are dynamic; if a recruiter does not include Aptitude/GD, the system automatically skips those stages.	
6	Scheduling Module	Recruiters assign specific time slots for Aptitude Tests, Group Discussions, and Interviews to shortlisted candidates. The Conflict-Detection Algorithm validates against all existing commitments.	MongoDB, Express.js
7	Conflict-Detection Algorithm	A custom MongoDB aggregation pipeline that actively validates proposed time slots against a student's existing commitments across all stages (tests, GDs, interviews), blocking double-books in real-time.	MongoDB Aggregation, Node.js
8	Semantic Resume Intelligence	A Python microservice that moves beyond TF-IDF. It uses Sentence-BERT to generate contextual embeddings of resumes and JDs, calculating similarity based on <i>meaning</i> rather than mere keywords.	Python (Flask), Sentence-Transformers, PyPDF2
9	Explainable AI (XAI)	Complements the Semantic Resume Engine. When a student is shortlisted or	Python, Sentence-BERT

	Feedback Module	rejected, the system displays the <i>specific sentences and semantic contexts</i> from their resume that positively or negatively influenced their match score. This ensures absolute transparency, builds student trust, and mitigates unconscious bias.	
10	Recruiter Result Declaration	Recruiters declare results at each stage: Aptitude (Pass/Fail), GD (Shortlist/Reject), and Interview (Select/Reject/Waitlist)—in bulk or individually.	MongoDB, Express.js
11	Offer Letter Management	Recruiters upload official offer letters (PDF) for selected students. The system auto-triggers a congratulatory email with a personalized download link. Students view/download the PDF and formally Accept or Decline the offer directly from their dashboard; decline reasons are recorded for TPO analytics.	Multer, MongoDB GridFS, Nodemailer
12	CSV Bulk Import	TPOs upload Excel/CSV files to instantly create accounts for hundreds of students (name, email, roll number, CGPA, branch, backlogs). The system auto-triggers activation emails.	Multer, csv-parser
13	Recruiter Trust Score	An automated background verification system that fetches WHOIS domain age, cross-verifies with MCA/ROC data,	WHOIS API, Node.js

		and generates a trust score for TPO approval.	
14	Automated Email Reminders	A node-cron scheduler that triggers Nodemailer to dispatch 24-hour reminders for Aptitude Tests, GDs, Interviews, and result declaration emails.	node-cron, Nodemailer
15	TPO Placement Analytics Dashboard	ECharts visualizations displaying: Placed vs. Unplaced students, branch-wise statistics, Stage-wise Funnel Visualization (Applications → Aptitude Cleared → GD Cleared → Interviewed → Placed), Offer-to-Acceptance Ratio , and Stage-wise Drop-off Analytics .	ECharts
16	Progressive Web App (PWA)	Service Workers and Web App Manifest enable offline access to cached schedules and dashboards, and app-like installation on mobile devices.	Service Workers, Workbox
17	Live System Alerts	Socket.IO -powered pop-up notifications when recruiters schedule tests/GDs/interviews or update application statuses.	Socket.IO
18	Localhost Deployment	Complete orchestration of Frontend (Angular), Backend (Node.js), AI Service (Python), and Database (MongoDB) on the local development machine.	npm, nodemon

4. Complete Application Lifecycle (Visual Flow)

This flowchart represents the definitive, end-to-end journey of a student's application through the multi-stage pipeline:



5. Tools & Technologies Used (Semester 1)

The project strictly adheres to 100% Free & Open-Source Software (FOSS) principles across all layers.

Layer	Technology Stack	Purpose
Frontend	Angular 17+, HTML5, CSS3, Tailwind/Bootstrap	Build responsive, mobile-first user interfaces for all 3 stakeholder roles.
Backend	Node.js, Express.js	RESTful API development, business logic, and server-side orchestration (Monolith).
AI & ML Layer	Python (Flask), Sentence-Transformers (all-MiniLM-L6-v2), PyTorch, PyPDF2	Semantic embedding generation, resume parsing, and XAI feature importance mapping.
Database	MongoDB, MongoDB GridFS	Flexible NoSQL storage for transactional data and large file storage (resumes, offer letters).
Real-Time Alerts	Socket.IO	Live pop-up notifications for status updates and scheduling events.
PWA & Offline Support	Service Workers, Workbox, Web App Manifest	Offline-first caching and app-like installability on mobile devices.
Authentication	JSON Web Tokens (JWT), bcrypt	Secure, stateless password hashing and session-less authentication.
Email Service	Nodemailer, node-cron	Automated 24-hour reminders and result declaration emails.
Analytics	ECharts	Interactive data visualizations for TPO dashboard.
Local Server	npm, nodemon, Python 3.9+, MongoDB Community	Local development and testing environment.

6. Pros and Cons of the Project

A nuanced, balanced analysis demonstrating acute awareness of the system's real-world capabilities and limitations.

Pros (Strengths & Novel Advantages)

SL.NO	Pro	Explanation
1	Eliminates Double-Booking Chaos	The custom MongoDB conflict-resolution pipeline actively prevents overlapping scheduling across Aptitude Tests, GDs, and Interviews in real-time.
2	Semantic AI Over Keyword Matching	By utilizing Sentence-BERT, the system understands contextual synonyms and nuanced skill descriptions, shortlisting candidates based on <i>actual capability</i> rather than arbitrary word matching.
3	Radical Transparency (XAI)	Unlike black-box AI systems, the Explainable AI module shows students and recruiters <i>exactly which sentences</i> drove their match score—building trust and ensuring fair, unbiased shortlisting.
4	Multi-Stage Recruitment Pipeline	Supports the complete real-world recruitment funnel—Aptitude Tests, Group Discussions, and Interviews—with dynamic status tracking.
5	Closes the Recruitment Loop	Unlike most systems that stop at scheduling, CampusPulse provides full closure with result declaration, offer letter uploads, and student accept/decline tracking—giving TPOs genuine final placement data.
6	Reduces Administrative Overhead	Automation of reminders, eligibility filtering, conflict detection, and offer tracking drastically reduces manual effort for TPOs.
7	100% FOSS	Entire stack is free and open-source, ensuring zero licensing costs.
8	PWA Offline Support	Students can view their schedules even without an internet connection.

Cons (Limitations & Future Challenges)

SL.NO	Con	Explanation & Mitigation Strategy
1	Relies on TPO Diligence for Verification	The Recruiter Trust Score is automated, but the final approval still relies on TPO oversight. <i>Mitigation:</i> Add a public rating/review system for recruiters in a future update.
2	SMTP Rate-Limiting	Automated reminders depend on Gmail SMTP, which has strict daily limits. <i>Mitigation:</i> Queue emails using Redis and implement a fallback SMTP server.
3	Initial Data Entry Overhead	While CSV import helps, initial student onboarding still requires some manual effort. <i>Mitigation:</i> Integrate OAuth-based LinkedIn/Google profile imports in Semester 2.
4	Semantic Model Bias	Sentence-BERT is trained on general corpus data. It may misclassify niche, domain-specific engineering jargon. <i>Mitigation:</i> Plan to fine-tune the model on a custom dataset of Indian campus resumes in future iterations.
5	No Native Mobile Application	PWA provides offline support, but lacks native push notifications (e.g., Firebase). <i>Mitigation:</i> Provide an optional progressive upgrade path for Firebase Cloud Messaging in later versions.
6	Offer Letter Forgery Risk	Uploaded PDFs could be forged. <i>Mitigation:</i> Integrate digital signatures (DSC) in future iterations.

7. Future Scope (Semester 2 – Main Project)

The following features are planned for the Main Project in Semester 2:

SL.NO	Enhancement	Description
-------	-------------	-------------

1	Private 1-on-1 Chat	Real-time private messaging between Students, Recruiters, and TPOs using Socket.IO .
2	Mass Broadcast	TPOs send emergency alerts to all students; Recruiters send targeted updates to their applicants.
3	Microservices Architecture	Refactor the monolithic backend into independent services (Auth, Drives, Scheduling, AI, Notifications).
4	Docker Containerization	Package the entire stack (FE, BE, AI, DB) into isolated containers managed via docker-compose.
5	CI/CD Pipeline	Automated testing and deployment triggered on every GitHub push using GitHub Actions.
6	Cloud Deployment	Deploy frontend on Vercel, backend on Render/Railway, and database on MongoDB Atlas.
7	Multi-Tenancy	Support multiple colleges with collegeId-based data isolation.
8	Research Paper Publication	Submit a full research paper to a National/International conference detailing the Semantic AI, XAI, and Multi-Stage Pipeline.

8. Conclusion

CampusPulse transforms the chaotic campus placement ecosystem into an intelligent, transparent, and efficient data-driven platform. By moving beyond simple automation and embracing **Semantic AI for merit-based shortlisting**, **Explainable AI for radical transparency**, and a **complete multi-stage recruitment pipeline with offer-to-acceptance lifecycle**, this project solves deep-rooted information asymmetry, operational friction, and communication gaps that plague higher education institutions.

The Mini Project successfully demonstrates the core workflow—from job drive creation to final offer acceptance—on a localhost deployment. The system reduces manual effort for TPOs, provides transparency for students, and delivers better-qualified candidates for recruiters. With the foundation laid in Semester 1, the Main Project will scale this prototype to an enterprise-grade, multi-tenant, cloud-deployed platform with microservices, containerization, and CI/CD pipelines.